第三章 栈和队列

3.1 栈（Stack）

1. 定义
2. 逻辑结构
3. 存储结构
4. 运算规则
5. 实现方式

3.2 队列（Queue）

1. 定义
2. 逻辑结构
3. 存储结构
4. 运算规则
5. 实现方式

第3章作业：严蔚敏教材配套习题集

3. 5　3. 6　3. 10　3. 21　3. 29　3. 32

3.2 队列

尾部插入

首部删除

队列定义 只能在表的一端进行插入运算，在表的另一端进行删除运算的线性表。

逻辑结构 与线性表相同，仍为一对一关系。

存储结构 顺序队或链队，以循环顺序队更常见。

运算规则 只能在队首和队尾运算，且访问结点时依照先进先出（FIFO）的原则。

实现方式 关键是掌握入队和出队操作，具体实现依顺序队或链队的不同而不同。

基本操作：入队或出队，建空队列，判队空或队满等操作。

队列（**Queue**）是仅在表尾进行插入操作，在表头进行删除操作的线性表。它是一种先进先出（ＦＩＦＯ）的线性表。

例如：队列 $Q=(a_1, a_2, a_3, \ldots\ldots\ldots, a_{n-1}, a_n)$

队首　　队尾

在队尾插入元素称为入队；在队首删除元素称为出队。

问：为什么要设计队列？它有什么独特用途？

答：

1. 离散事件的模拟（**模拟事件发生的先后顺序,例如CPU芯片中的指令译码队列**）；
2. 操作系统中的作业调度（**一个CPU执行多个作业**）；
3. 简化程序设计。

队的实现方式是本节重点，**关键是掌握入队和出队操作。具体实现依存储结构（链队或顺序队）的不同而不同。**

1. 链队列　　2. 顺序队

重点是循环顺序队

队的抽象数据类型定义：

ADT Queue{

数据对象：D=……

数据关系：R=……

基本操作：

……

} ADT Queue

建队、入队或出队、判队空或队满等，教材P59-60罗列了9种基本操作。

1. 链队列

因简单而先介绍

结点类型定义：

```
typedef Struct QNode{
    QElemType      data;     //元素
    Struct  QNode  *next;  //指向下一结点的指针
  }Qnode , * QueuePtr ;
```

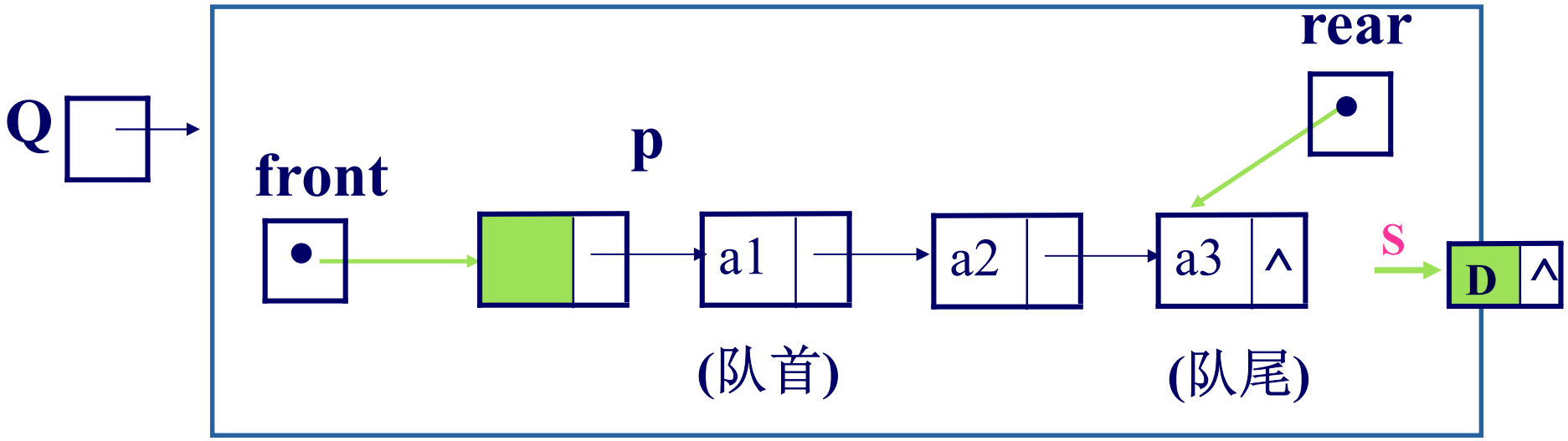
链队中任一结点的结构

链队列类型定义：

```
typedef  struct {
  QueuePtr     front ; //队首指针
  QueuePtr     rear ; //队尾指针
  }  LinkQueue;
```

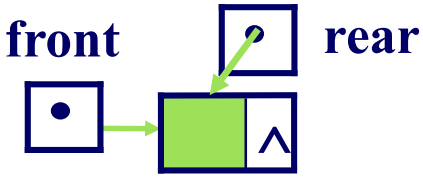
关于整个链队的总体描述

链队示意图：



讨论：

① 空链队的特征？ **front=rear**



② 链队会满吗？一般不会，因为删除时有**free**动作。除非内存不足！

③ 怎样实现链队的入队和出队操作？

入队（尾部插入）： **rear->next=S; rear=S;**

出队（头部删除）： **front->next=p->next;**

完整操作函数见教材P62下

课堂练习：

（1） 在一个链队中，假设f和r分别为队首和队尾指针，则插入s所指结点的运算时_____。

A. f->next=s;f=s;　　B. r->next=s; r=s; √

C. s->next=r;r=s;　　D. s->next=f; f=s;

（2） 在一个链队中，假设f和r分别为队首和队尾指针，则删除一个结点的运算时_____。

A. r=f->next;　　B. r=r->next;

C. f=f->next; √　　D. f=r->next;

2. 顺序队

顺序队类型定义：

关于整个顺序队的总体描述

```
#define   QUEUE-MAXSIZE  100  //最大队列长度
   typedef   struct {
        QElemType    *base;   //队列的基址
         int              front;   //队首指针
         int               rear;   //队尾指针
    }SqQueue
```

顺序队中每个结点的结构描述在此省略，是QElemType类型。

建队核心语句：

```
q . base=(QElemType *)malloc(sizeof (QElemType ）
* QUEUE_MAXSIZE);      //分配空间
```

采用动态分配空间的形式

顺序队示意图：

讨论：

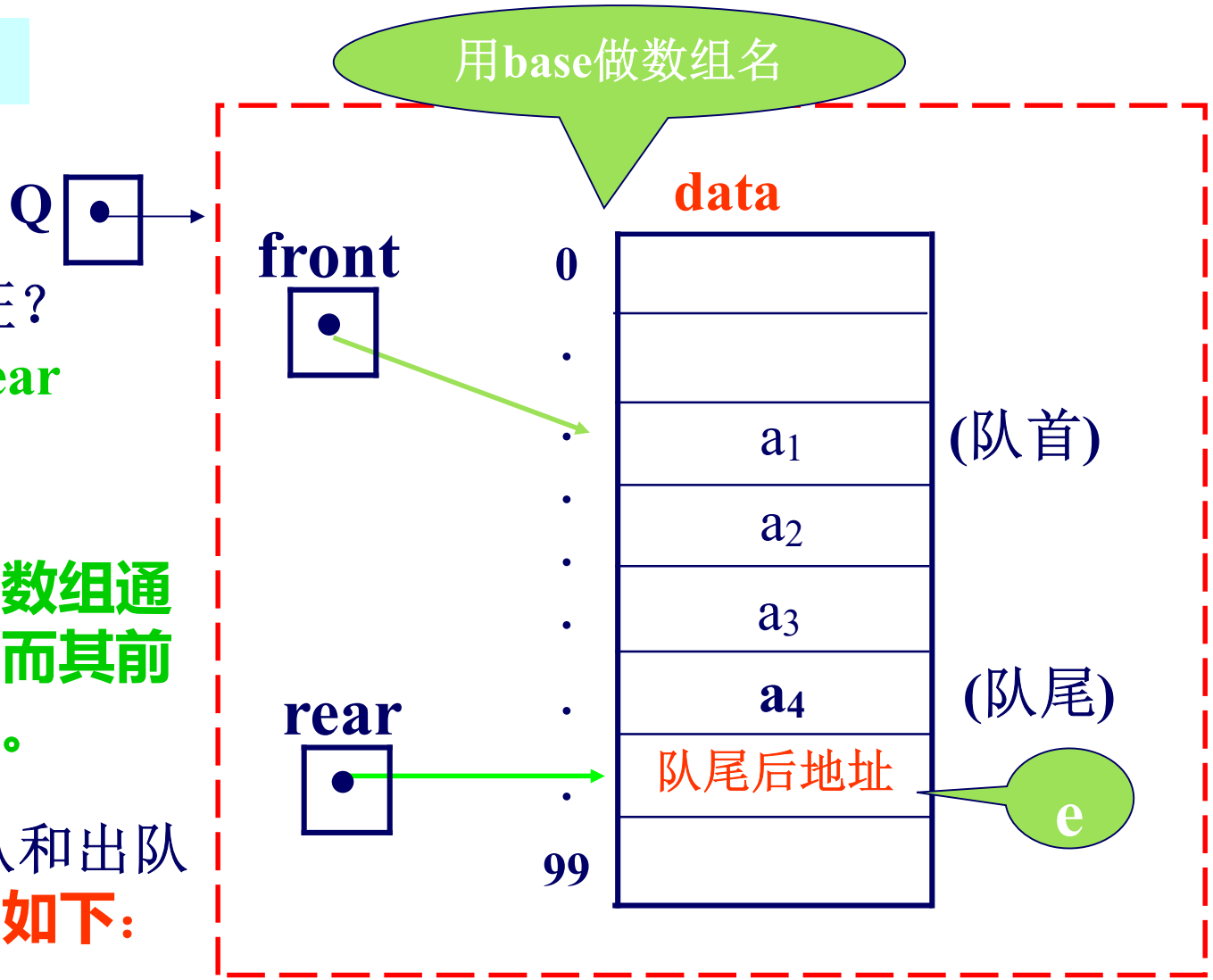
① 空队列的特征？

约定：**front=rear**

② 队列会满吗？

极易装满！因为数组通常有长度限制，而其前端空间无法释放。

③ 怎样实现入队和出队操作？**核心语句如下：**



入队(尾部插入)： **Q[rear]=e; rear++ ;**

出队(头部删除)： **e=Q[front]; front++;**

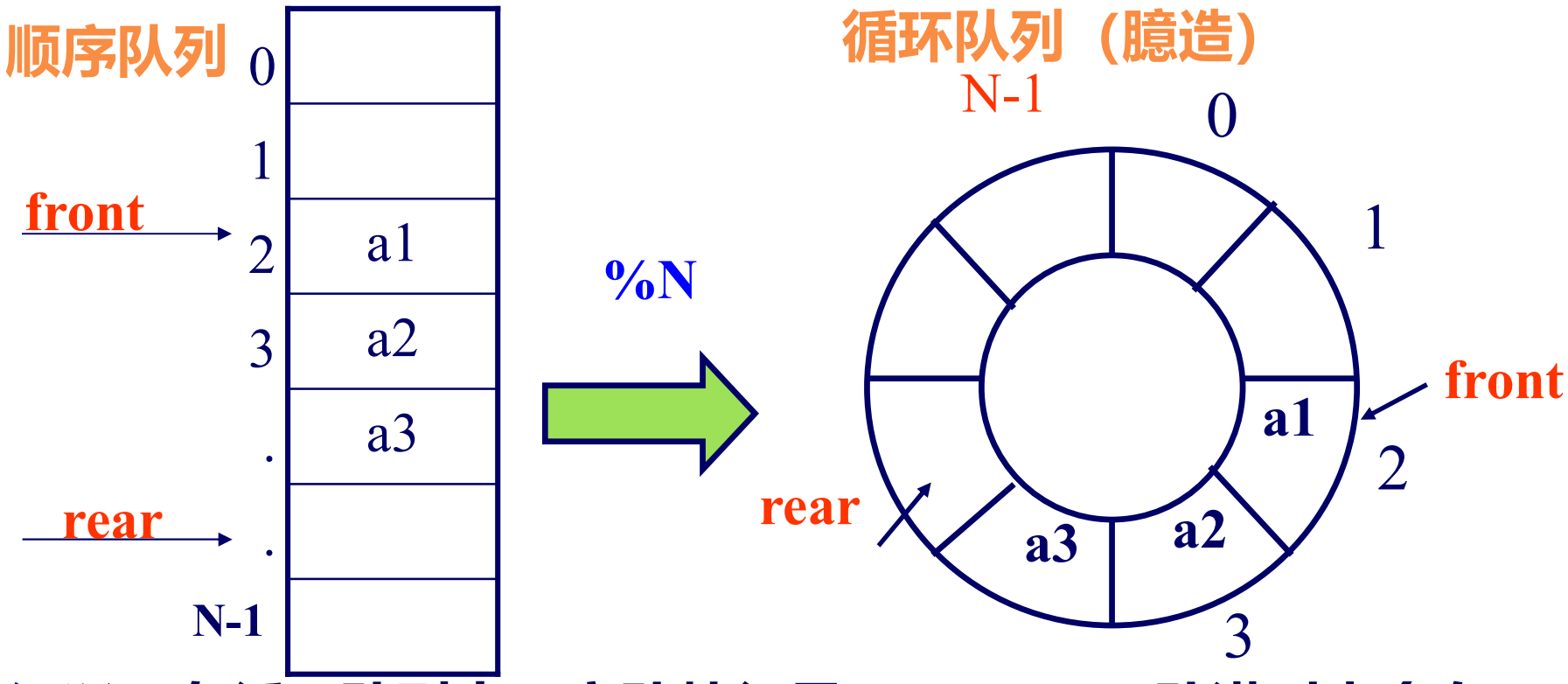
有缺陷

假溢出！

问：什么叫“假溢出”？如何解决？

答：**在顺序队中，当尾指针已经到了数组的上界，不能再有入队操作，但其实数组中还有空位置，这就叫“假溢出”。**

解决假溢出的途径——采用循环队列



新问题：**在循环队列中，空队特征是front=rear；队满时也会有front=rear；判决条件将出现二义性！**

解决方案有三：

①使用一个计数器记录队列中元素个数（即队列长度）；

②加设标志位，**删除时置1,插入时置0,则可识别当前front=rear属于何种情况**

③ 人为浪费一个单元，则队满特征可改为**front=(rear+1)%N；**

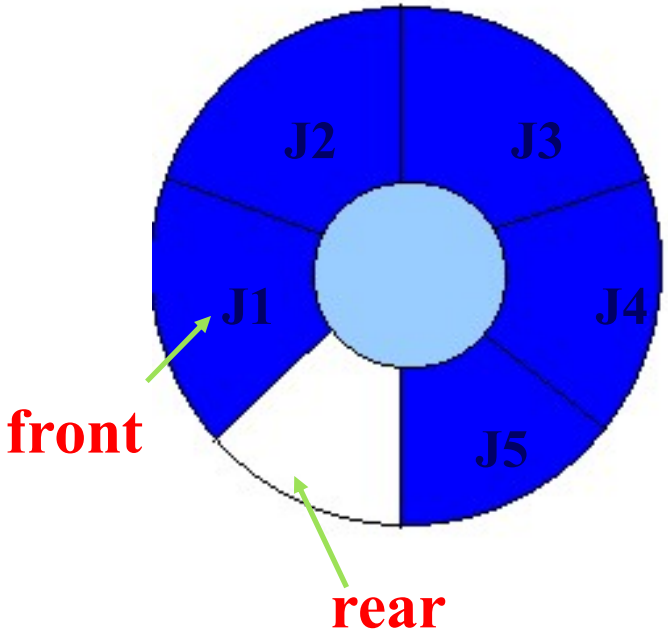
实际中常选用**方案3**（人为浪费一个单元）：

即front和rear二者之一指向实元素，另一个指向空闲元素。

队空条件：**front = rear**　　(初始化时：**front = rear**)

队满条件：　**front = (rear+1) % N**　　**(N=maxsize)**

队列长度（**即数据元素个数**）：**L=（N＋rear－front）% N**



问1：左图中队列**maxsize N=**？**6**

问2：左图中队列长度**L=**？　**5**

问3：　在具有**n**个单元的循环队列中，队满时共有多少个元素？

N-1个

例1：数组Q [n]用来表示一个循环队列，f 为当前队列头元素的前一位置，r 为队尾元素的位置。假定队列中元素的个数小于n，计算队列中元素的公式为:

（A） r－f　　（B） （n＋f－r）% n

（C）n＋r－f　　（D） （n＋r－f）% n

√

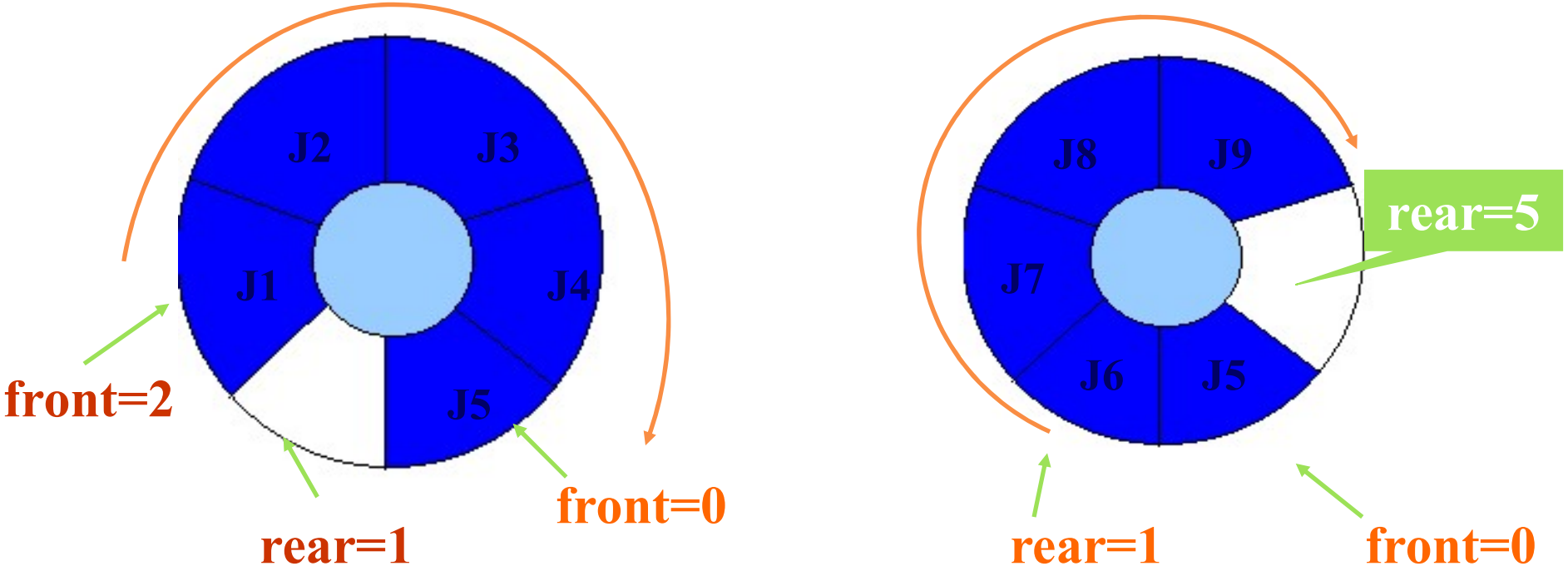
答：　**要分析4种公式哪种最合理?**

当 r ≥f 时（A）合理；
当 r < f 时（C）合理；

综合2种情况，以（D）的表达最为合理

例2：在一个循环队列中，若约定队首指针指向队首元素的前一个位置。那么，从循环队列中删除一个元素时，其操作是 先 移动队首指针 ，后 取出元素 。

例3： 一循环队列如下图所示，若先删除4个元素，接着再插入4个元素，请问队头和队尾指针分别指向哪个位置?



解：由图可知，队头和队尾指针的初态分别为front=2和rear=1。

删除4个元素后， f= (2+4) %6=0

再插入4个元素后，r=(1+4)%6=5

讨论：循环队列的基本操作如何实现？

以建队、入队和出队三种基本操作为例

1）初始化一个空队列

算法要求：生成一空队列

算法操作：为队列分配基本容量空间

设置队列为空队列，其特征即：

front=rear=0（也可为任意单元，如**1**，**2**，**…**甚至**-1** ）

建队的完整算法：

```
Status   InitQueue ( SqQueue    &q ) //初始化空循环队列 q
{    q . base=(QElemType *)malloc(sizeof(QElemType）
* QUEUE_MAXSIZE);   //分配空间
if (!q.base)  exit(OVERFLOW);//内存分配失败，退出程序
  q.front =q.rear=0; //置空队列
   return OK;
 } //InitQueue;
```

2） 入队操作

算法说明：向循环队列的队尾插入一个元素

分析：

(1) 插入前应当先判断队列是否满？

```
if (( q . rear + 1 ) % QUEUE_MAXSIZE )==q.front)
  return ERROR;
```

(2）插入动作

```
q.base [q.rear] = e;
q.rear = ( q . rear + 1 ) %  QUEUE_MAXSIZE;
```

队列尺寸

入队操作完整算法

```
Status   EnQueue(SqQueue  &q,   QElemType e)
{//向循环队列 q 的队尾加入一个元素 e
  if (   (q.rear+1) %  QUEUE_MAXSIZE = =  q.front  )
              return  ERROR ; //队满则上溢，无法再入队
    q.base [ q.rear ] = e;    //新元素e入队
 q.rear = ( q . rear + 1 ) %  QUEUE_MAXSIZE; //指针后移
    return  OK;
}// EnQueue;
```

rear指针在元素入队后再加

3）出队操作

算法说明：删除队首元素，返回其值 e

分析：

（1） 在删除前应当判断队列是否空？

```
if (q.front = q.rear ) return ERROR;
```

（2）删除动作分析；

前面约定指针front指向队首元素的位置，故：

```
e = q.base [ q.front ] ;
q.front = ( q.front + 1 ) % QUEUE_MAXSIZE;
```

front指针在元素出队后再加

队列尺寸

出队操作完整算法

```
Status    DeQueue ( SqQueue  &q,   QElemType &e)
{//若队列不空，删除循环队列q的队头元素，
    //由 e 返回其值，并返回OK
    if ( q.front = = q.rear )   return ERROR;//队列空
    e = q.base [ q.front ] ;
    q.front=(q.front+1) % QUEUE_MAXSIZE ;
    return OK;
}// DeQueue
```

例：对4个数据元素进行队操作。在进队操作时，按a1、a2、a3、a4次序每次进入一个元素（假设队的初态为空）。

对这4个数据元素进行的队操作是进队2次，出队1次，再进队2次，出队1次；这时，第1次出队得到的元素是 a1 第2次出队得到的元素是 a2 。经操作后，最后在队中的元素还有 2 个，队尾指针指向 v[0]

解：此题用V[4]大小数组即可（V[0]～V[3] 4个单元有效）。

① 进队2次　a1、a2　f=r=0 ;　r++ ; r++　(f=0, r=2)

② 出队1次　a1　f++;　(f=1, r=2)

③ 再进队2次　a2、a3、a4　r++ ; r++　(f=1, r=4=0)

④ 出队1次　a2　f++　(f=2, r=0)

本章小结

线性表、栈、队的异同点：

相同点：**逻辑结构相同，都是线性的；都可以用顺序存储或链表存储；栈和队列是两种特殊的线性表，即受限的线性表（只是对插入、删除运算加以限制）。**

不同点：

① 运算规则不同：

- **线性表为随机存取；**
- **而栈是只允许在一端进行插入和删除运算，因而是后进先出表LIFO；**
- **队列是只允许在一端进行插入、另一端进行删除运算，因而是先进先出表FIFO。**

② 用途不同，线性表比较通用；堆栈用于函数调用、递归和简化设计等；队列用于离散事件模拟、OS作业调度和简化设计等。(以后再举例)

第3章
结束

下周一交第三章作业

第3章作业： 严蔚敏教材配套习题集

3.5　3.6　3.10　3.21　3.29　3.32